

DOM и события

JavaScript в браузере

Когда JS выполняется в браузере, добавляются три модуля:

- **ECMAScript (ES)** — стандарт, описывающий сам язык JavaScript (синтаксис, встроенные объекты, функции). ES5, ES6/ES2015, ES2017 и т.д. — версии этого стандарта.
- **DOM (Document Object Model, объектная модель документа)** — интерфейс для работы с HTML-документом: поиск, создание, удаление элементов, реакция на действия пользователя.
- **BOM (Browser Object Model, объектная модель браузера)** — интерфейс для работы с самим браузером: размеры окна, прокрутка, URL, история навигации и т.п.

HTML-документ и DOM

DOM представляет HTML-документ как дерево. Каждый узел — JS-объект.

- **Element node (узел-элемент)** — создаётся тегом.
- **Text node (текстовый узел)** — создаётся текстом внутри элемента. Не может иметь дочерних элементов (всегда на нижнем уровне иерархии). Пробелы и переносы строк — тоже текстовые узлы.

Каждый элемент (кроме `document`, он корневой) вложен в один родительский. Браузер строит DOM постепенно, по мере получения кода — специальный модуль **HTML-парсер** превращает HTML в DOM.

Исключения при построении дерева:

- пробелы перед `head` игнорируются;
- всё, что идёт после `body`, браузер переносит внутрь `body`.

Поиск элементов

Основной способ — методы `querySelector*` (ищут по CSS-селектору).

```
element.querySelector(selector) // первый подходящий элемент, либо null
element.querySelectorAll(selector) // псевдомассив всех подходящих, либо пустой массив
```

Доступ к DOM начинается с объекта `document` — он часть глобального объекта `window`.

Навигация по DOM

Термины отношений: предок (**ancestor**), потомок (**descendant**), родитель (**parent**), дочерний элемент (**child**), сосед (**sibling**). Корневой элемент — **root node**.

Свойство	Что возвращает
<code>elem.parentNode</code>	родительский узел
<code>elem.childNodes</code>	все дочерние узлы (элементы + текст)
<code>elem.children</code>	только дочерние элементы (по тегам)
<code>elem.firstChild</code> / <code>elem.lastChild</code>	первый/последний дочерний узел (любой)
<code>elem.firstElementChild</code> / <code>elem.lastElementChild</code>	первый/последний дочерний элемент
<code>elem.previousSibling</code> / <code>elem.nextSibling</code>	соседний узел (любой)
<code>elem.previousElementSibling</code> / <code>elem.nextElementSibling</code>	соседний элемент

На практике эти свойства используются редко — вложенность часто меняется динамически, код становится ненадёжным. Знать стоит для понимания структуры DOM.

Свойства и атрибуты

Стандартные HTML-атрибуты становятся свойствами DOM-объекта и доступны через точку. Их можно читать и менять — изменения сразу отражаются в DOM.

```
const link = document.querySelector(".link");
console.log(link.href); // "https://goit.global"
link.href = "https://neo.goit.global";
```

textContent

Возвращает (и позволяет менять) весь текст внутри элемента, включая вложенные элементы. Что бы ни присвоили — всегда запишется как текст.

```
const el = document.querySelector(".text");
console.log(el.textContent); // "Username: Mango"
el.textContent = "Username: Poly";
```

classList

Объект, похожий на массив (но не массив), хранящий CSS-классы элемента. Есть `value` (строка класса) и `length` (кол-во классов), но чаще используются методы:

Метод	Действие
<code>classList.contains(name)</code>	есть ли класс — <code>true</code> / <code>false</code>
<code>classList.add(name)</code>	добавить класс (можно несколько через запятую)
<code>classList.remove(name)</code>	удалить класс (если его нет — без ошибки)
<code>classList.toggle(name)</code>	добавить, если нет / убрать, если есть
<code>classList.replace(old, new)</code>	заменить один класс на другой

Имя класса передаётся строкой без точки (без селектора класса).

Свойство style

Свойство `style` читает и меняет инлайн-стили (**inline styles**) элемента. Возвращает объект — экземпляр интерфейса `CSSStyleDeclaration` (декларация стилей CSS), содержащий только инлайн-свойства элемента, а не весь CSS.

Названия свойств записываются в **camelCase** (кэмел-кейс) вместо дефисов: `background-color` → `backgroundColor`.

```
const button = document.querySelector(".btn");

button.style.backgroundColor = "teal";
button.style.fontSize = "24px";
button.style.textAlign = "center";
```

На практике стилизация обычно делается через CSS-классы. Свойство `style` нужно для динамических стилей, когда значение заранее неизвестно (например, фон приходит с бэкенда).

Доступ к атрибутам

Методы работают со значением, которое находится в HTML-атрибуте.

Метод	Действие
<code>element.hasAttribute(name)</code>	есть ли атрибут — <code>true</code> / <code>false</code>
<code>element.getAttribute(name)</code>	значение атрибута, либо <code>null</code>
<code>element.setAttribute(name, value)</code>	установить/изменить значение атрибута
<code>element.removeAttribute(name)</code>	удалить атрибут (если его нет — без ошибки)

Некоторые атрибуты доступны и напрямую как свойства DOM-объекта (короче по коду) — как в примерах с `href` и `style` выше. А для удаления или проверки наличия удобнее использовать методы.

Собственные атрибуты (data-атрибуты)

data-атрибуты (data attributes) позволяют добавить к тегу произвольный атрибут и получить его значение в JavaScript — например, чтобы хранить тип действия кнопки.

```
<button type="button" data-action="save">Save text</button>
```

Значение читается через свойство `dataset` — приставка `data-` отбрасывается:

```
const saveBtn = document.querySelector('button[data-action="save"]');
console.log(saveBtn.dataset.action); // "save"
```

Изменение и добавление — как у обычного свойства объекта, удаление — оператором `delete`:

```
saveBtn.dataset.action = "update"; // изменить
saveBtn.dataset.role = "admin";    // добавить новый
delete saveBtn.dataset.action;      // удалить
```

data-атрибуты в HTML пишут в кебаб-кейсе (**kebab-case**), а в `dataset` они превращаются в кэмел-кейс: `data-user-role` → `dataset.userRole`.

Создание и удаление элементов

DOM API (Document Object Model Application Programming Interface, программный интерфейс объектной модели документа) позволяет выбирать, менять, удалять и создавать элементы.

```
const heading = document.createElement("h1");
heading.classList.add("title");
heading.textContent = "This is a heading";
```

`createElement(tagName)` создаёт элемент только в памяти — в DOM его ещё нет.

Чтобы элемент появился на странице, его нужно добавить одним из методов (вызываются на элементе `elem`, к которому добавляем):

Метод	Куда добавляет
<code>elem.append(e11, e12, ...)</code>	после всех детей <code>elem</code>
<code>elem.prepend(e11, e12, ...)</code>	перед всеми детьми <code>elem</code>
<code>elem.after(e11, e12, ...)</code>	после самого <code>elem</code>
<code>elem.before(e11, e12, ...)</code>	перед самым <code>elem</code>

Если добавляемый элемент уже есть в DOM, он переносится с прежнего места на новое — один и тот же элемент не может находиться в двух местах одновременно.

Удаление — метод `element.remove()`, вызывается на самом удаляемом элементе.

Свойство `innerHTML`

`innerHTML` хранит содержимое элемента вместе с тегами в виде строки. Доступно и для чтения, и для записи — при записи строки с HTML-тегами браузер парсит её и создаёт реальные элементы в DOM.

Запись пустой строки в `innerHTML` — быстрый способ полностью очистить содержимое элемента.

Минус: новое значение `innerHTML` полностью пересоздаёт всех потомков элемента. Если элемент не был пустым, это лишние затраты. Использовать `innerHTML` для добавления стоит только когда элемент пуст или нужно полностью заменить его содержимое.

Метод `insertAdjacentHTML()`

Современный метод добавления HTML-строки без пересоздания уже существующего содержимого элемента.

```
element.insertAdjacentHTML(position, string)
```

`position` — позиция вставки относительно `element`:

Значение	Позиция
<code>"beforebegin"</code>	перед <code>element</code>
<code>"afterbegin"</code>	внутри <code>element</code> , перед всеми детьми
<code>"beforeend"</code>	внутри <code>element</code> , после всех детей
<code>"afterend"</code>	после <code>element</code>

`"beforebegin"` и `"afterend"` работают, только если `element` уже находится в DOM.

События

Событие (event) — сигнал от браузера о том, что на странице что-то произошло (клик, нажатие клавиши, изменение размера окна и т.д.). Одно действие пользователя может вызывать несколько событий в фиксированном порядке — например, клик мышью: `mousedown` → `mouseup` → `click`.

Чтобы элемент реагировал на событие, к нему добавляют **слушателя события** (event listener) — механизм ожидания события — и обработчик.

```
element.addEventListener(event, handler, options)
```

- `event` — строка с именем события, например `"click"`

- `handler` — функция-колбэк, вызывается при наступлении события
- `options` — необязательный объект с доп. настройками

```
const button = document.querySelector(".my-button");

const handleClick = () => {
  console.log("Button clicked");
};

button.addEventListener("click", handleClick);
```

Лучше использовать именованную функцию, а не анонимную — это читаемее и позволяет позже удалить обработчик через `removeEventListener`. На одном элементе может висеть сколько угодно обработчиков, даже одного типа события — они вызываются в порядке регистрации.

Метод `removeEventListener()`

Удаляет слушателя события. Аргументы те же, что у `addEventListener` — важно передать ту же функцию-обработчик, которая была туда передана.

```
button.removeEventListener("click", handleClick);
```

Объект события

Каждое событие — это объект (все события происходят от базового класса `Event`), который содержит детали события и автоматически передаётся первым аргументом в обработчик. Называть параметр можно как угодно, обычно — `e`, `evt` или `event`.

```
const handleClick = event => {
  console.log(event);
};

button.addEventListener("click", handleClick);
```

Полезные свойства объекта события: `event.type` — тип события, `event.currentTarget` — элемент, на котором висит обработчик.

События клавиатуры

Две основные: `keydown` — клавишу нажали, `keyup` — клавишу отпустили. Объекты этих событий происходят от класса `KeyboardEvent`.

В отличие от других событий, события клавиатуры обрабатываются на `document`, а не на конкретном элементе.

```
document.addEventListener("keydown", event => {
  console.log(event.key, event.code);
});
```

`keydown` / `keyup` срабатывают на любой клавише, включая служебные (`Ctrl`, `Shift`, `Escape` и т.д.). На практике чаще обрабатывают только `keydown` — она срабатывает раньше.

Свойство	Что возвращает
<code>event.key</code>	символ с учётом модификаторов (<code>Shift</code>) и языка раскладки
<code>event.code</code>	код физической клавиши — не зависит от языка и модификаторов

Устаревшее свойство `keyCode` ещё встречается в старых статьях — вместо него используй `key` и `code`.

Клавиши-модификаторы

Для обработки сочетаний клавиш (например, `Ctrl + S`) на объекте события есть булевы свойства: `ctrlKey`, `altKey`, `shiftKey`, `metaKey` — показывают, была ли зажата соответствующая клавиша.

```
document.addEventListener("keydown", event => {
  if ((event.ctrlKey || event.metaKey) && event.code === "KeyS") {
    console.log("Ctrl+S or Cmd+S");
  }
});
```

Некоторые сочетания конфликтуют со встроенным поведением браузера (например, `Ctrl+D` — добавление в закладки) — старайся не пересекаться с ними.

События элементов форм

submit

Событие `submit` происходит на самой форме (тег `form`) — при клике на кнопку `type="submit"` или при `Enter` в любом текстовом поле формы.

```
const form = document.querySelector("form");

form.addEventListener("submit", event => {
  event.preventDefault();
});
```

Многие события запускают действие браузера по умолчанию (клик по ссылке — переход, отправка формы — перезагрузка страницы). Метод `event.preventDefault()` отменяет это действие.

Свойство `elements` формы содержит все её поля с атрибутом `name` — удобно для валидации: `event.target.elements.login.value`.

change

Происходит после изменения значения элемента формы.

- для текстовых полей и `textarea` — только после потери фокуса, не на каждый ввод символа;
- для `select`, чекбоксов и радиокнопок — сразу при выборе значения.

input

Происходит только на текстовых полях и `textarea`, срабатывает при каждом изменении значения (ввод/удаление символа), не дожидаясь потери фокуса. Основное событие для работы с текстовыми полями.

Событие	Когда срабатывает
<code>input</code>	при каждом изменении значения, фокус не важен
<code>change</code>	после потери фокуса, если значение изменилось

focus и blur

Элемент получает фокус по клику или переходом клавишей `Tab`.

- `focus` — элемент получил фокус;

- `blur` — элемент потерял фокус (например, клик в другом месте).

Фокус можно ставить/снимать программно методами `focus()` и `blur()`. Элемент, находящийся в фокусе сейчас, — `document.activeElement`.

Фокус может быть только на одном элементе одновременно. Большинство неинтерактивных элементов (например, `<div>`) фокус получить не могут.